SHMEM入门：共享内存的概念、价值与使用

时间：2026/2/28

https://gitcode.com/cann/shmem

CANN

SHMEM发展历程：25年发布，支持AICore直驱mte/rdma，逐步覆盖通信/通算类主要使用场景模板样例

里程碑

2025-930

- 支持AICore直驱MTE/RDMA实现
- 提供ag、ar等算子样例

2025-1230

- Gitcode开源
- CANN社区SHMEM-SIG运营
- 新增shuffleKV、mm+rs、ag+mm等进阶算子样例

2026-330

- 支持AICore直驱SDMA实现
- AICore直驱MTE扩展支持950平台

2026-430

- 支持AICore直驱UDMA实现

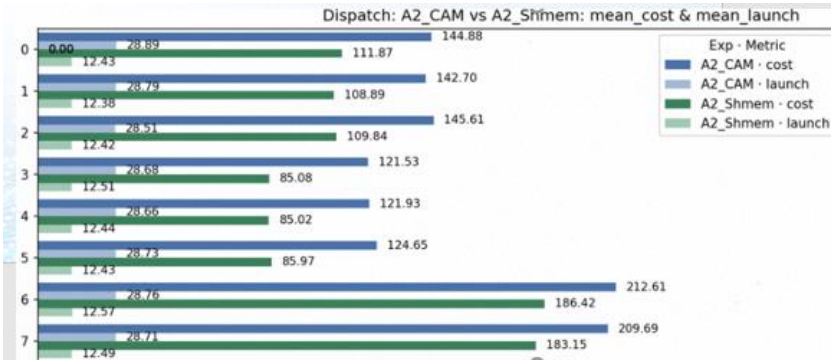
2025年HC大会正式发布

生态伙伴贡献
联合共建CANN生态

贡献者列表

- 华南理工大学 陆璐教授团队

产业落地多家企业
支撑定制算子开发，实现高性能提升



https://gitcode.com/cann/shmem

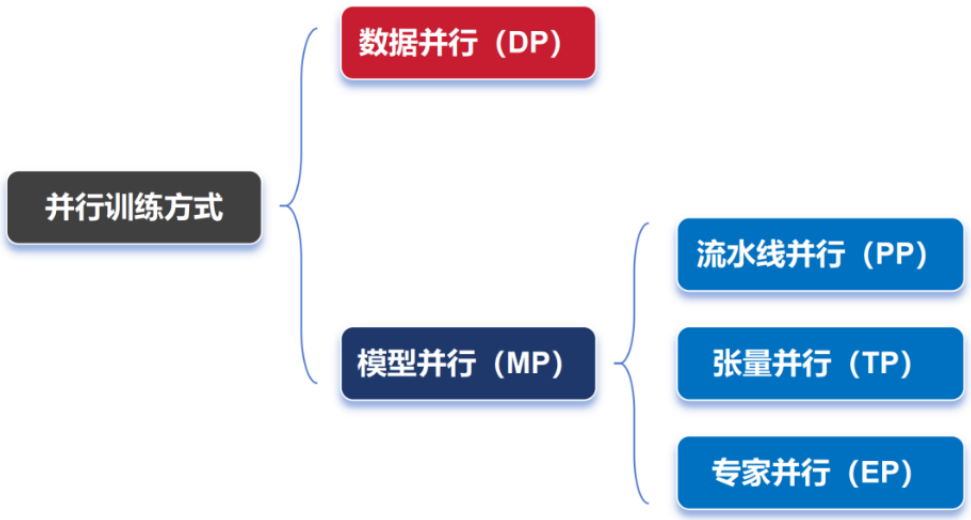
CANN

应用场景

场景分类	典型应用场景	硬件组网
场景1：NPU集群内共享内存访问	• 单边通信算子、通信库 • 通算融合算子（DeepEP、Dispatch & Combine）	• RDMA • HCCS • UB Memory、URMA
场景2：NPU 发起访问CPU 共享内存池	• LLM推理KVCache Context Offloading • Diffusion多模态 DiTCache Offloading • 推荐 Embedding	• HCCS • UB Memory、URMA

为什么需要通信?

- 对于小模型而言，虽然单卡能放下模型，但可能没法放下一个batch所要的全部数据。此时需要数据并行。
- 对于大模型而言，单卡无法放下一个模型，必须将模型切分到多张卡上。此时需要模型并行。



通信是分布式训练 / 推理的基础，提供分布式扩展的基础能力。
通算融合算子可以通过计算与通信并行掩盖延迟、提升硬件利用率，实现“边算边传”。

https://gitcode.com/cann/shmem

CANN

背景

- 分布式AI性能瓶颈凸显：通信计算耦合紧、时延敏感度高、内存墙问题严峻
- 现有集合通信原语应对细粒度通算融合、Tile级编程范式诉求时开发难度高

使用封装好的集合通信原语进行细粒度优化难度高，开发自定义通算/通信算子上手难度大

难点1 硬件层：多架构 / 互联拓扑的适配复杂度高：

- 多种底层通信硬件：MTE，xDMA等多种通信引擎如何协同
- 拓扑结构差异：单机多卡，多机集群，超节点等多种拓扑的通信逻辑兼容

难点2 多领域知识交叉学习难度大：

- 开发者需要同时掌握硬件指令，Kernel 优化，通信协议设计，分布式框架适配等多项技术
- 同步 / 异步调度的精细化控制难度大，不同并行策略的通信量、粒度不同，优化方向（带宽优先 / 延迟优先）难以平衡。

核心诉求：高效开发高性能定制算子

通算&通信类算子开发难点

- 多机多卡初始化复杂
- CPU调度时延高
- 偏移计算难度大
- 集合通信粒度较大
- 学习底层指令困难
- 通信引擎实现差距大

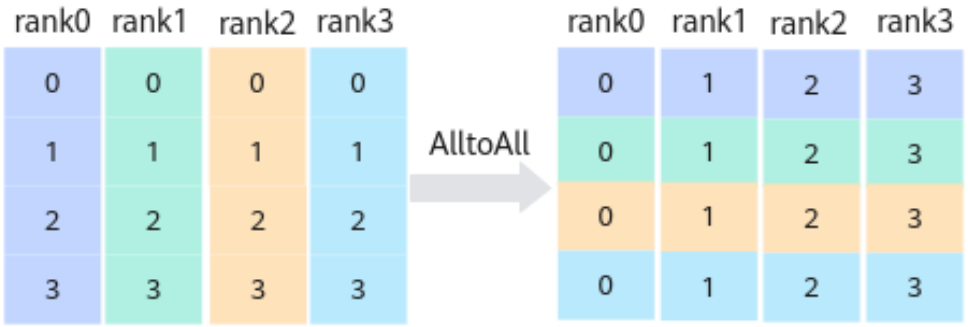
核心目标

- 简洁的多卡访存
- 简化开发指令
- 高性能低时延

解决方案

- 全局统一地址空间
- 单边内存语义
- 集成多种高性能通信引擎
- NPU直驱访存

高效开发高性能定制算子



https://gitcode.com/cann/shmem

CANN

SHMEM 共享内存库简介

项目背景：提供高效、易用的AICore直驱访存能力，支撑全局共享内存构建、通算融合算子创新。简化卡间数据同步与通算融合算子开发流程。

业界定位：SHMEM 是面向昇腾平台的多机多卡内存通信库

提供Host侧与Device侧接口，实现跨设备的高效内存访问与数据同步。

设计理念：

- Host/Device双侧接口体系，支持Host与Device协同工作流。
- 高性能通信优化，内置MTE、xDMA引擎支持，实现远程内存直接读写，减少数据传输延迟，简化开发流程。

https://gitcode.com/cann/shmem

CANN

SHMEM基本架构：Host/Device双侧接口体系

接口设计贴合昇腾算子开发范式，支持Host与Device协同工作流：

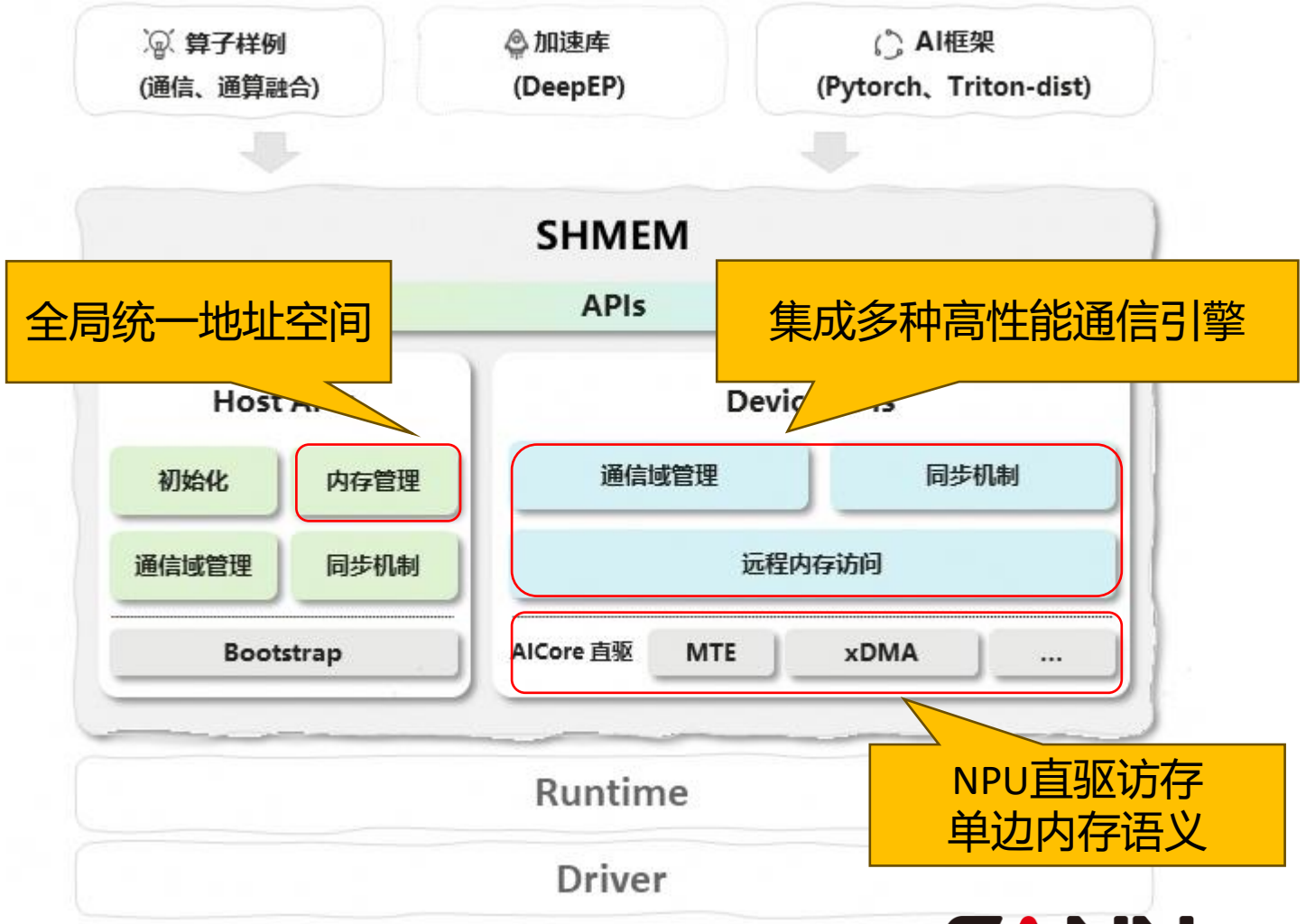
- **Host层：** 负责初始化、内存堆管理、通信域（Team）创建及全局同步。

  提供多种host侧Bootstrap方案，兼容第三方(如MPI)通信框架，支持host基本集合通信能力

- **Device层：** 提供远程内存访问（RMA）、设备级同步及通信域操作。

内置MTE、xDMA引擎支持，实现远程内存直接读写，减少数据传输延迟

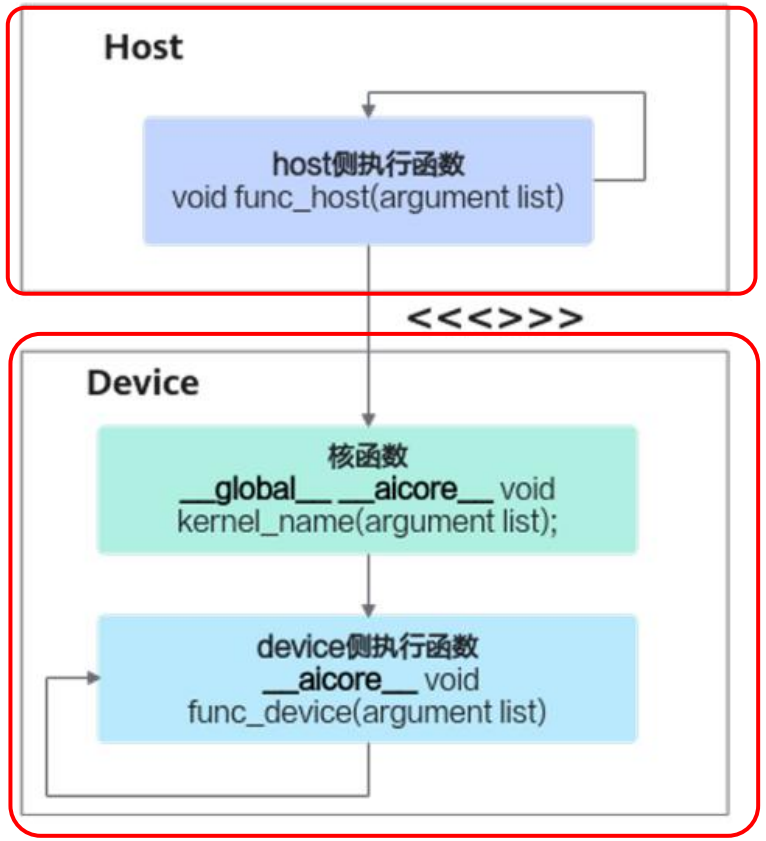
针对昇腾硬件特性优化数据传输路径，提升多卡协同效率。



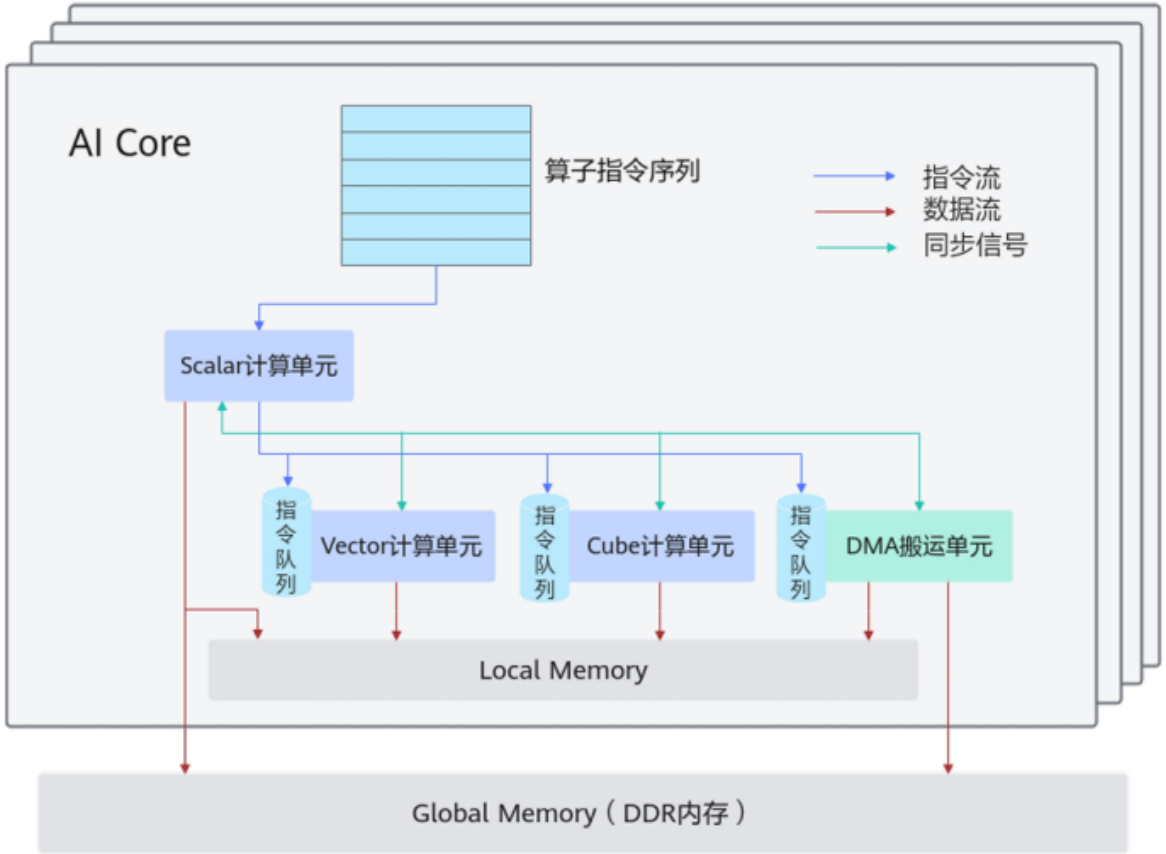
https://gitcode.com/cann/shmem

CANN

Ascend平台上的Host和Device如何交互



异构编程模式



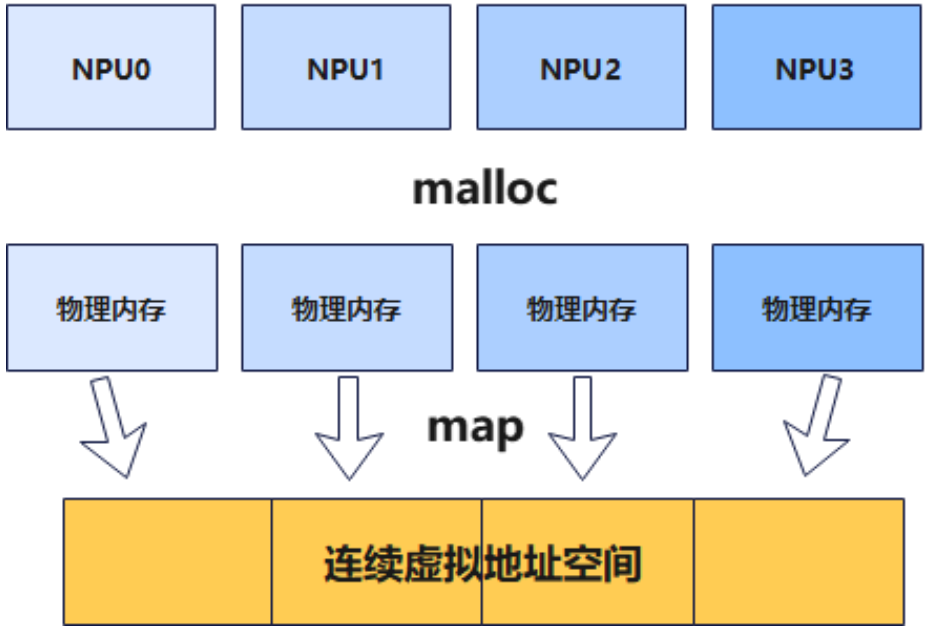
AICore抽象硬件架构

https://gitcode.com/cann/shmem

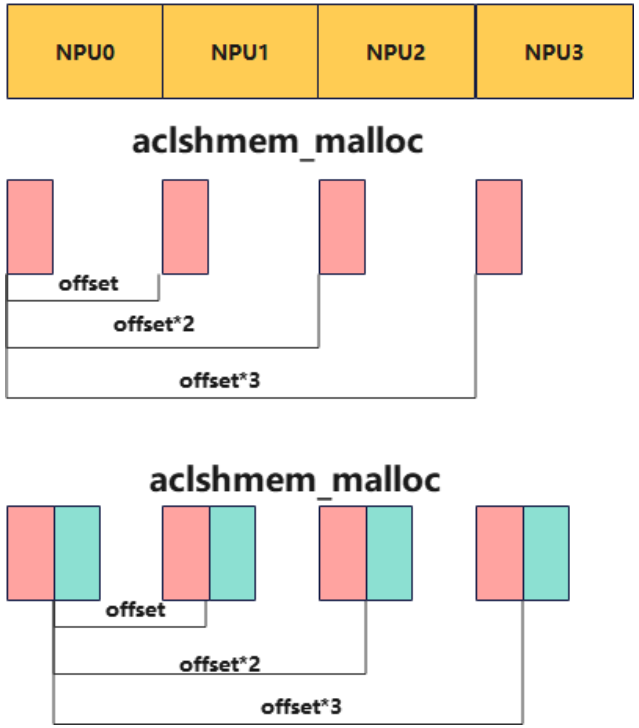
CANN

SHMEM重要特性：对称内存管理

SHMEM 将多个 NPU 的物理地址统一整合，并映射至一整块连续的虚拟地址空间。借助地址偏移，能在任意 NPU 上快速访问 SHMEM 域内任意目标NPU的目标地址信息。



所有PE的虚拟地址空间一致



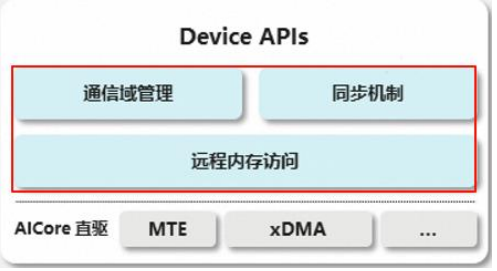
卡间地址偏移固定

一致的虚拟地址空间便于开发者理解算子
固定的地址偏移便于开发者快速执行跨卡访问与写入

https://gitcode.com/cann/shmem

CANN

SHMEM重要特性：更细粒度的通信操作



编写定制化通信操作，集成多种高性能通信引擎

SHMEM分别提供了**高阶**与**低阶**的RMA接口

高阶接口：仅感知源地址、目标地址、搬移元素数量和目标pe，**自动选择**底层通信引擎(开发更易用)

```
ACLSHMEM_DEVICE void aclshmem_##NAME##_get_nbi(__gm__ TYPE *dst, __gm__ TYPE *src, uint32_t elem_size, int32_t pe)
```

低阶接口（以MTE为例）：额外感知ub信息，底层通信引擎，同步TEventID信息(性能更极致)

```
ACLSHMEM_DEVICE void aclshmemx_mte_get_nbi(__gm__ T *dst, __gm__ T *src, __ubuf__ T *buf, uint32_t ub_size, uint32_t elem_size, int pe, AscendC::TEventID EVENT_ID)
```

SHMEM分别提供了**通信域**和**p2p**级别的barrier接口

通信域级别barrier，全局等待(开发更易用)

```
ACLSHMEM_DEVICE void aclshmem_barrier_all(void)
```

p2p级别barrier，等待目标地址PE flag就绪(逻辑更灵活)

```
ACLSHMEM_DEVICE int32_t aclshmem_signal_wait_until(__gm__ int32_t *sig_addr, int cmp, int32_t cmp_val);
```

CANN

https://gitcode.com/cann/shmem

SHMEM共享内存库算子开发更便捷

跨卡数据搬移更便捷

SHMEM实现gm2gm搬运

```
aclshmemx_mte_put_nbi(dst_gm, src_gm, tmp_buff, ub_size,
element_size, pe, EVENT_ID);
```

https://gitcode.com/cann/shmem/blob/master/examples/allgather/allgather_kernel.cpp

SHMEM get

DataSize/核	多核平均 Time/us
16B	0.70505
32B	0.700825
64B	0.699858
128B	0.702442
256B	0.751346
512B	0.822921
1024B	0.851592
2KB	0.918754
4KB	1.252563
8KB	2.429033
16KB	4.826938
32KB	9.475913
64KB	18.84581
128KB	37.54237

DataCopy

DataSize/核	多核平均 Time/us
16B	0.728633
32B	0.701683
64B	0.701796
128B	0.702158
256B	0.746471
512B	0.817567
1024B	0.849925
2KB	0.902204
4KB	1.165467
8KB	2.340558
16KB	4.689554
32KB	9.351725
64KB	18.75823
128KB	37.51603

核数：48核 单用例循环 100次

DataCopy实现gm2gm搬运

```
TPipe pipe;
TQue<TPosition::VECIN, 1> inQueueSrc;
TQue<TPosition::VECOUT, 1> outQueueDst;
GlobalTensor<half> srcGlobal, dstGlobal;
pipe.InitBuffer(inQueueSrc, 1, 512 * sizeof(half));
pipe.InitBuffer(outQueueDst, 1, 512 * sizeof(half));
LocalTensor<half> srcLocal = inQueueSrc.AllocTensor<half>();
LocalTensor<half> dstLocal = outQueueDst.AllocTensor<half>();
DataCopy(srcLocal, srcGlobal, 512);
DataCopy(dstLocal , srcLocal, 512);
DataCopy(dstGlobal, dstLocal, 512);
```

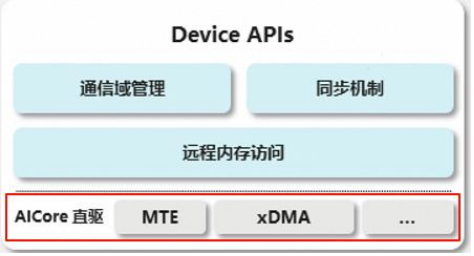
高易用：代码量降低50%+，性能基本无损耗

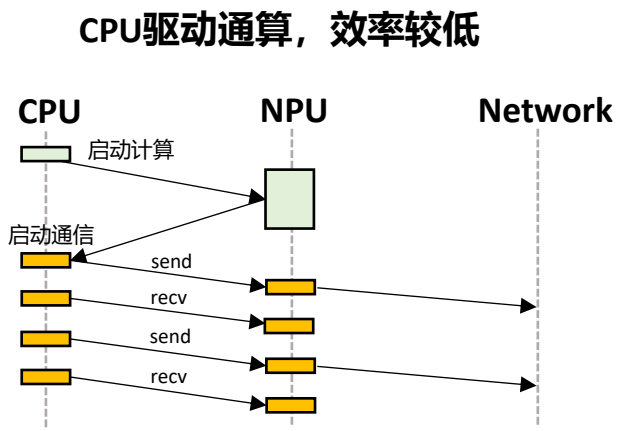
	shmem 行数	datacopy 行数	平均节省
计算 + 通信	728	1550	53.0%
通信	572	1230	53.5%
搬运接口	14.5（两种情况平均）	22	34.1%
同步接口	1	47	97.9%
初始化	10	1216	99.1%

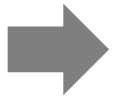
CANN

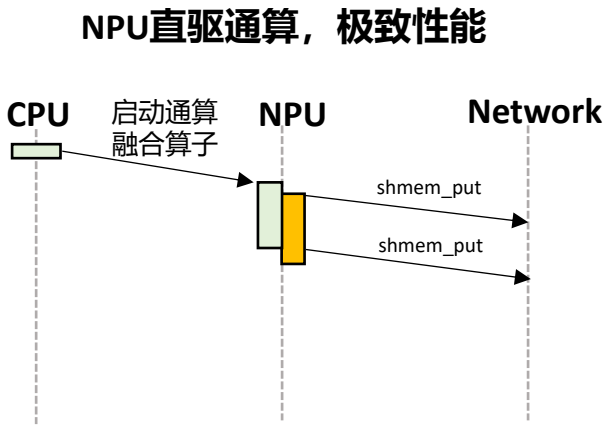
https://gitcode.com/cann/shmem

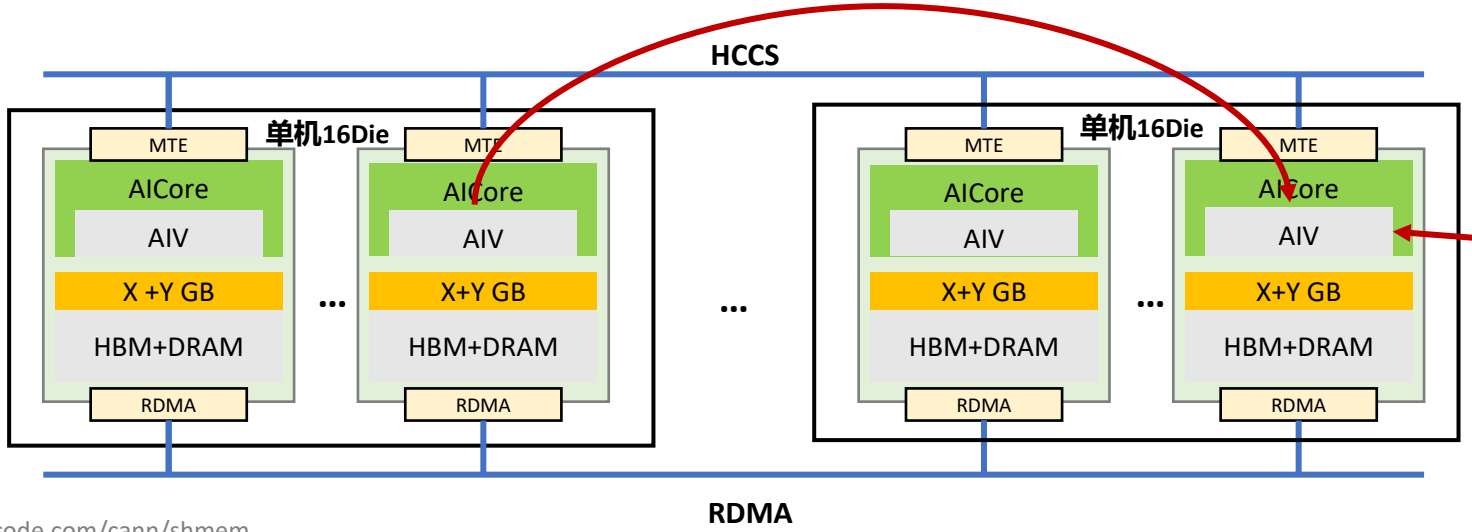
SHMEM重要特性：AICORE直驱的单边访存能力











KernelOp

访存（单边操作更简洁）

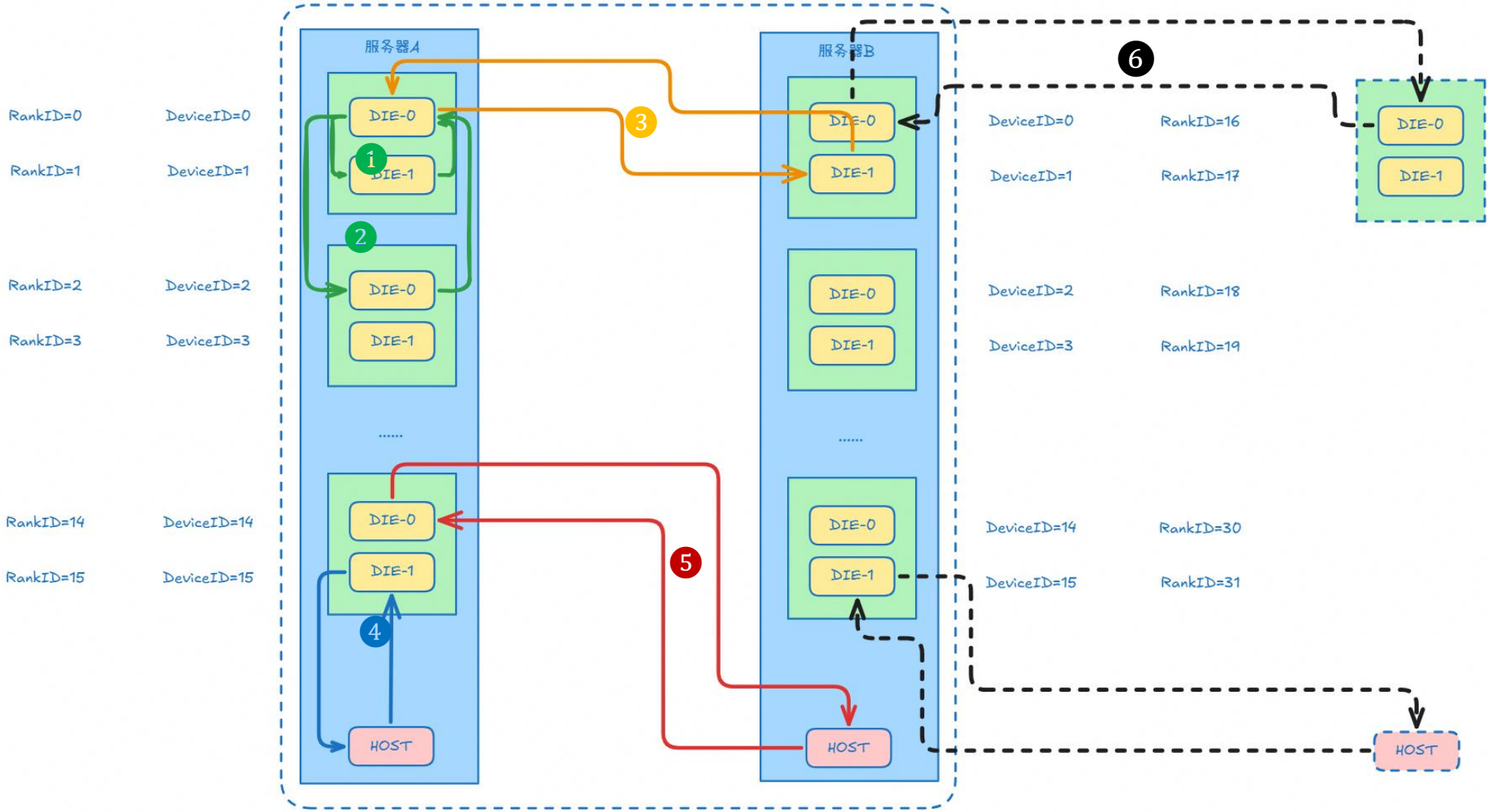
aclshmem_put_##NAME##_mem_xxxn

aclshmem_get_##NAME##_mem_xxx

https://gitcode.com/cann/shmem

CANN

SHMEM统一内存访问：打通单机 跨机链路，使能集群内D2D D2H通信能力



- **链路1：**
  - 单机D2D SIO
- **链路2：**
  - 单机D2D HCCS
- **链路3：**
  - 跨机D2D HCCS
- **链路4：**
  - 单机D2H HCCS
- **链路5：**
  - 跨机D2H HCCS
- **链路6：**
  - 跨超D2D Roce

CANN

快速上手

◻ 环境准备，安装NPU驱动固件以及CANN开发套件包

执行如下命令安装驱动。

```
./Ascend-hdk-<chip_type>-npu-driver_<version>_linux-<arch>.run --full --install-for-all
```

执行如下命令安装固件。

```
./Ascend-hdk-<chip_type>-npu-firmware_<version>.run --full
```

安装软件包。

```
./Ascend-cann-toolkit_<version>_linux-<arch>.run --install
```

◻ 使能CANN环境变量

配置环境变量，当前以非root用户安装后的默认路径为例，请用户根据set_env.sh的实际路径执行如下命令。

```
source ${HOME}/Ascend/ascend-toolkit/set_env.sh
```

CANN

快速上手

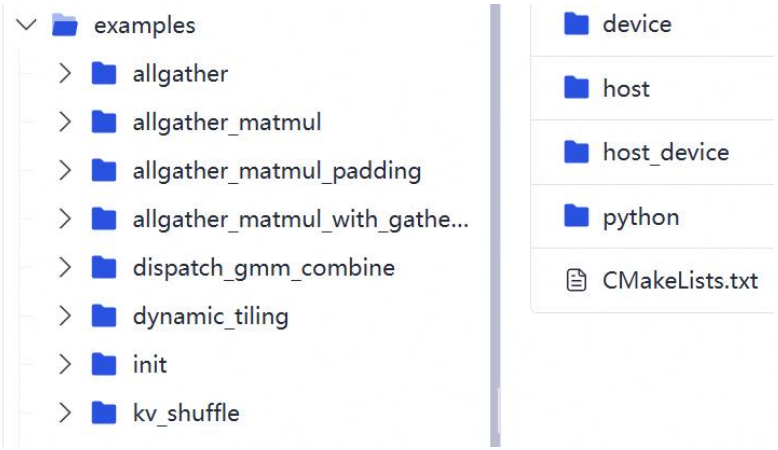
- 拉取代码到本地：git clone https://gitcode.com/cann/shmem.git

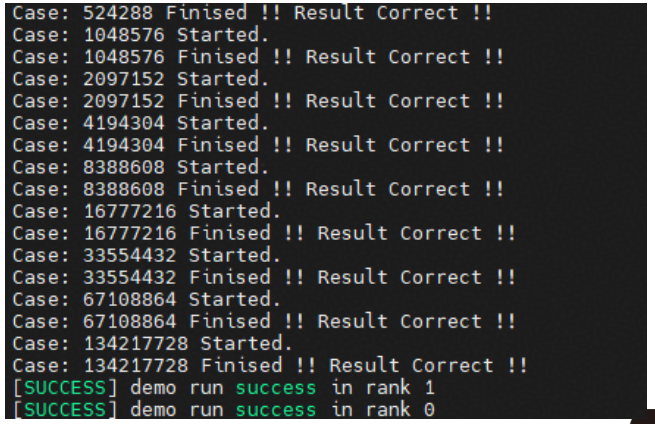
- 算子编译，使用cd命令进入shmem目录，执行： bash scripts/build.sh –examples

- 使能SHMEM环境变量 source install/set_env.sh

- 算子执行。

以allgather算子为例， cd命令进入shmem/examples/allgather，运行bash run.sh -ranks ${RANKS}脚本，RANKS设置为你想要拉起的卡数即可快速拉起算子用例。其他算子可以参考各自目录下的README文件执行。





https://gitcode.com/cann/shmem

参考链接：https://gitcode.com/cann/shmem/blob/master/docs/example.md

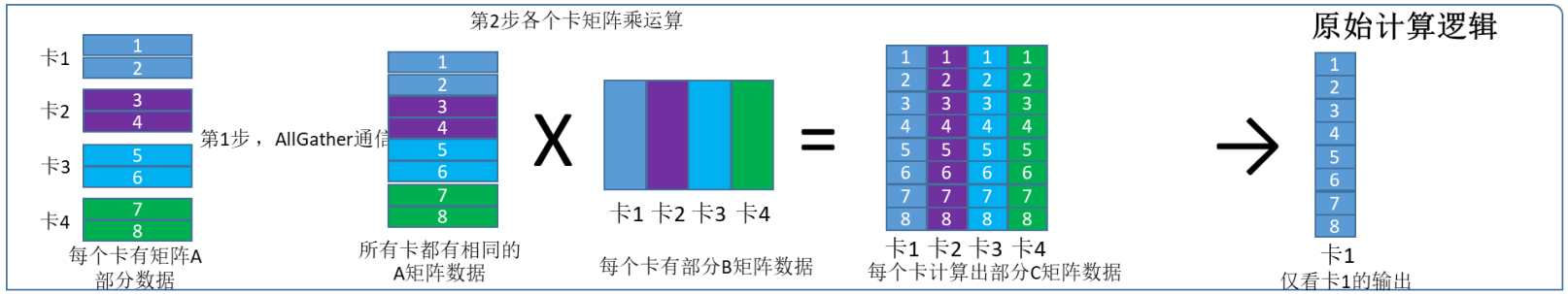
SHMEM应用场景

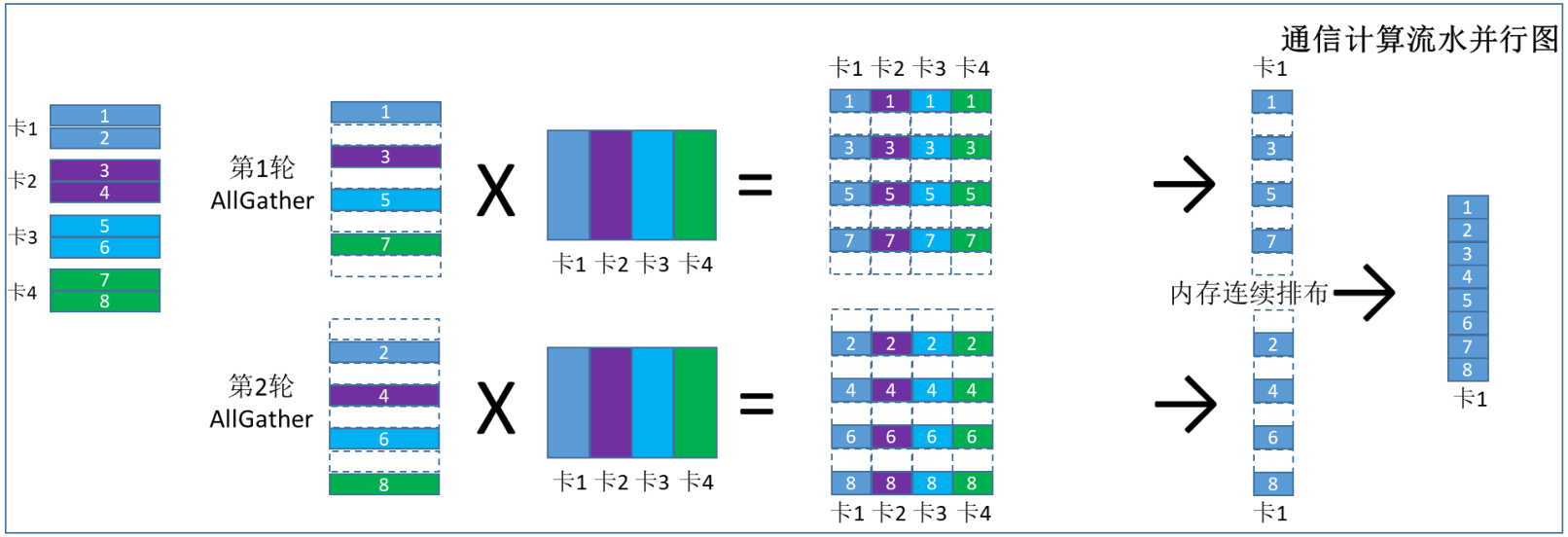
场景分类	典型应用场景	硬件组网
场景1：NPU集群内共享内存访问	• 单边通信算子、通信库 • 通算融合算子（DeepEP、Dispatch & Combine）	• RDMA • HCCS • UB Memory、URMA
场景2：NPU 发起访问CPU 共享内存池	• LLM推理KVCache Context Offloading • Diffusion多模态 DiTCache Offloading • 推荐 Embedding	• HCCS • UB Memory、URMA

CANN

场景1：基于D2D能力实现高灵活性的通算融合算子

- AllGatherMatmul样例

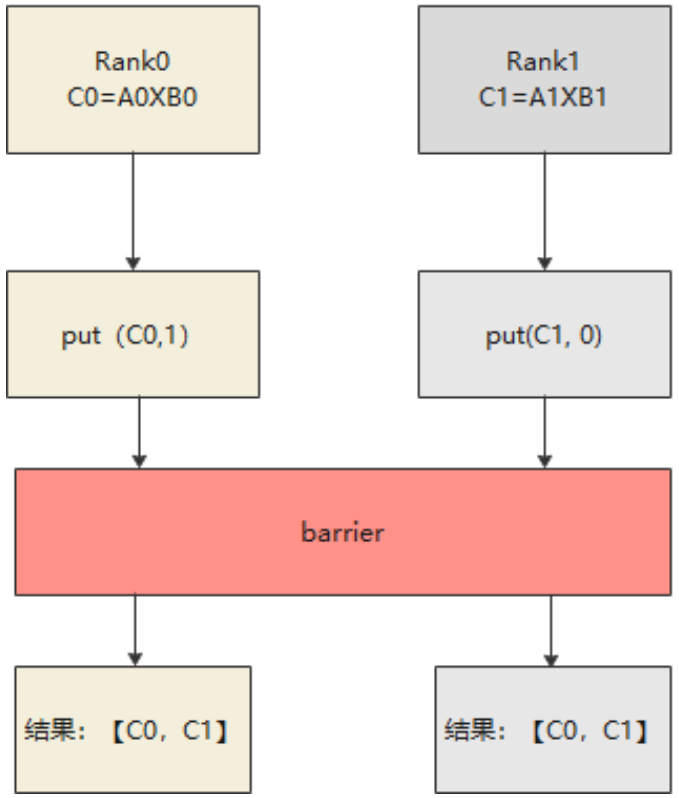




基于shmem实现的Kernel实现步骤：

- Host侧使用aclshmemx_init_attr创建对称内存堆，完成shmem 初始化
- Host侧使用aclshmem_malloc分配对称内存
- 将对称内存传递给kernel入参
- Device使用put/get实现allgather+matmul计算流水

场景1：基于D2D能力实现高灵活性的通算融合算子



通信部分

```
// Catcoc::DGemm::Kernel::MatmulReduceScatter, AIV部分
commLoops = (blockMatmulM * blockMatmulN) / blockPerComm
for (int block = 0; block < commLoops; block++) {
    WaitComputeFinish();
    // matmul与communication的block shape可以不一致（通常为了性能是不一致的）
    blockPerComm = mmadBlockToCommBlock(blockPerComm);
    // AIVBlockTile循环在AIVs上并行
    for (int blockInComm = 0; blockInComm < blockPerComm; blockInComm += A1

        // Catcoc::CommEpilogue::Block
        for (int tile = 0; tile < tileLoops; tile++) {

            // Catcoc::CommEpilogue::Tile
            // TileMmad使用指令aclshmem_mte_get/put_nbi
            for (int tileRepeat = 0; tileRepeat < r; tileRepeat++) {
                for (int tileLength = 0; tileLength < l; tileLength++) {
                    dataCopy.call(dst, src);
                } // tileLength
            } // tileRepeat

        } // comm mainloop

    } // blockInComm
    SetCommunicationFinish();
} // block, pipeline stages
```

计算部分

```
// Catcoc::DGemm::Kernel::MatmulReduceScatter, AIC部分
commLoops = (blockMatmulM * blockMatmulN) / blockPerComm
for (int block = 0; block < commLoops; block++) {
    WaitCommunicationFinish();
    // AICBlockTile循环在AICs上并行
    for (int blockInComm = 0; blockInComm < blockPerComm; blockInComm += AICBlockTile) {

        // Catlass::Gemm::Block::BlockMmad: Catlass BlockMmad, 在kTile上迭代
        // BlockMmad输出到shmem
        for (int kTile = 0; kTile < blockMatmulK; kTile++) {

            // Catlass::Gemm::Tile::TileMmad
            // TileMmad使用指令Ascend::Mmad
            for (int tileMmaM = 0; tileMmaM < m; tileMmaM++) {
                for (int tileMmaN = 0; tileMmaN < n; tileMmaN++) {
                    for (int tileMmaK = 0; tileMmaK < k; tileMmaK++) {
                        mmad.call(c, a, b);
                    } // tileMmaK
                } // tileMmaN
            } // tileMmaM

        } // kTile mainloop

    } // blockInComm
    SetComputeFinish();
} // block, pipeline stages
```

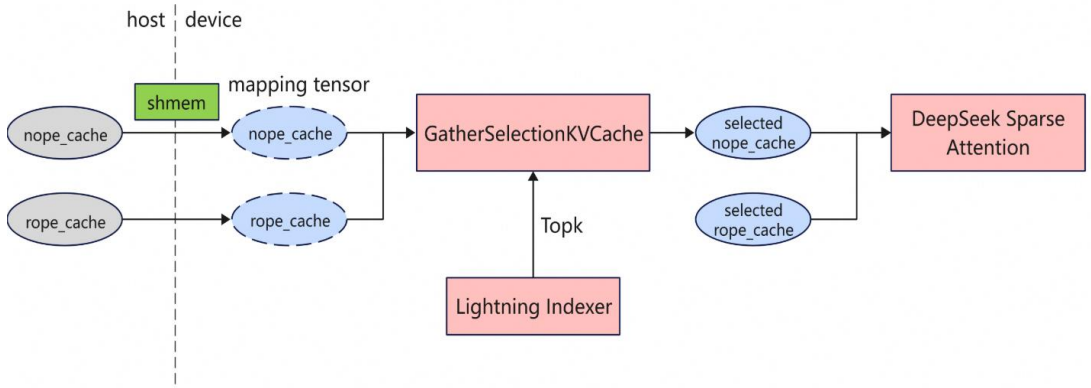
CANN

场景2：基于D2H优化长序列稀疏Attention+KVCache Offload

- **统一内存管理：shmem透明化Host-Device访存差异，简化数据搬运。**
- **shmem 低延迟高带宽通信加速 KV Cache Offload，提升推理性能。**

- 在模型初始化阶段，调用 shmem 为将要卸载的 KV Cache 分配 Host 侧内存，并将其地址映射到 Device

```
cache_nope_bytes = cache_nope_tensor.untyped_storage().nbytes()
cache_nope_data_ptr = aclshmem_malloc(cache_nope_bytes, MemType.HOST_SIDE)
metadata = {
    "data_ptr": cache_nope_data_ptr,
    "device": cache_device,
    "nbytes": cache_nope_bytes,
    "dtype": dtype,
    "size": cache_nope_shape,
    "stride": cache_nope_tensor.stride(),
    "storage_offset": cache_nope_tensor.storage_offset(),
}
storage = torch_npu._C._construct_storage_from_data_pointer(
    metadata["data_ptr"], metadata["device"], metadata["nbytes"]
)
cache_nope = torch_npu._C._construct_NPU_Tensor_From_Storage_And_Metadata(metadata, storage)
```



- 在 Prefill 阶段，模型推理生成的 KV Cache 会被异步卸载到这块内存中；
- 在 Decoding 阶段，Lightning Indexer 生成稀疏注意力的 topk ids 之后，调用 GatherSelectionKVCache 算子直接操作映射后的 Device tensor，无需感知其在 Host 侧的内存排布，简化了算法流程。

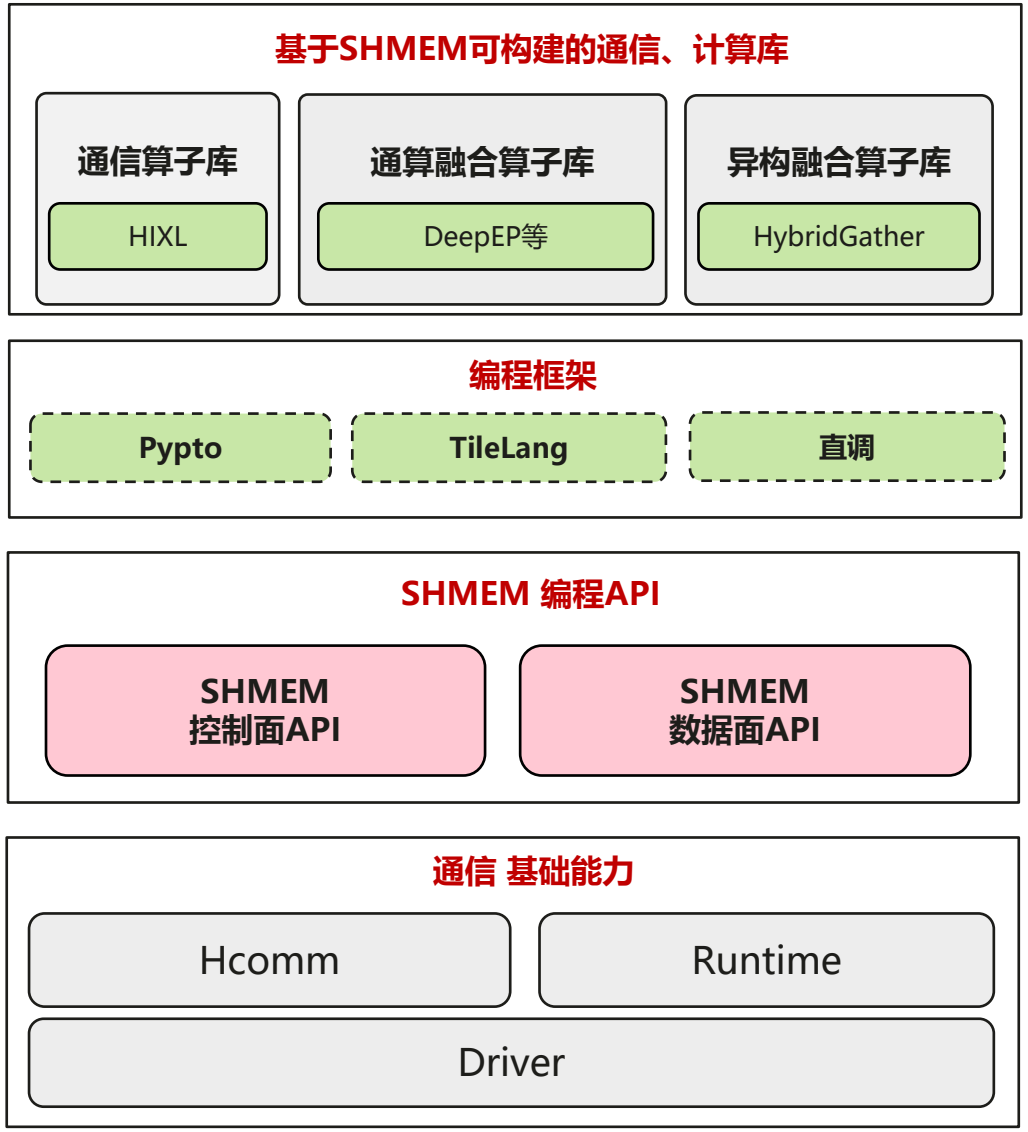
```
# kv cache constructed by shmem and mapped into device tensor
full_kv_cache = k_nope.squeeze(2)
full_k_rope = k_pe.squeeze(2) if not self.kv_cache_c8 else offload_cache.empty_rope

selection_kv_actual_seq = torch_npu.npu_gather_selection_kv_cache(
    selection_k_rope, selection_kv_cache,
    selection_kv_block_table, selection_kv_block_status,
    topk_indices, full_k_rope, full_kv_cache,
    block_table, actual_seq_lengths_kv,
    actual_seq_qlen, selection_topk_block_size=1)
```

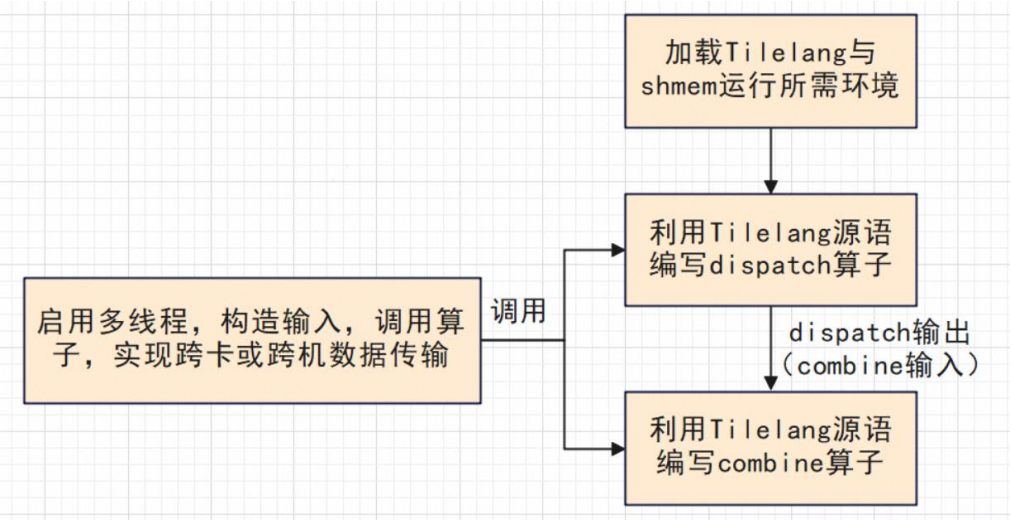
使能 Offload + shmem 后，在 DeepSeek V3.2 上取得了明显的推理内存收益：**基于 Atlas A3 64卡环境，64K 序列长度下，模型推理支持的最大 batch size 可以从 1024 增长到 2048**

CANN

SHMEM与CANN编程框架,通信库深度配合，高度匹配多层级，各领域业务场景



- TileLang中封装 OPENSHMEM 部分通信语义，Tilelang用户可直接调用Tilelang-shmem。算子实现流程如下：



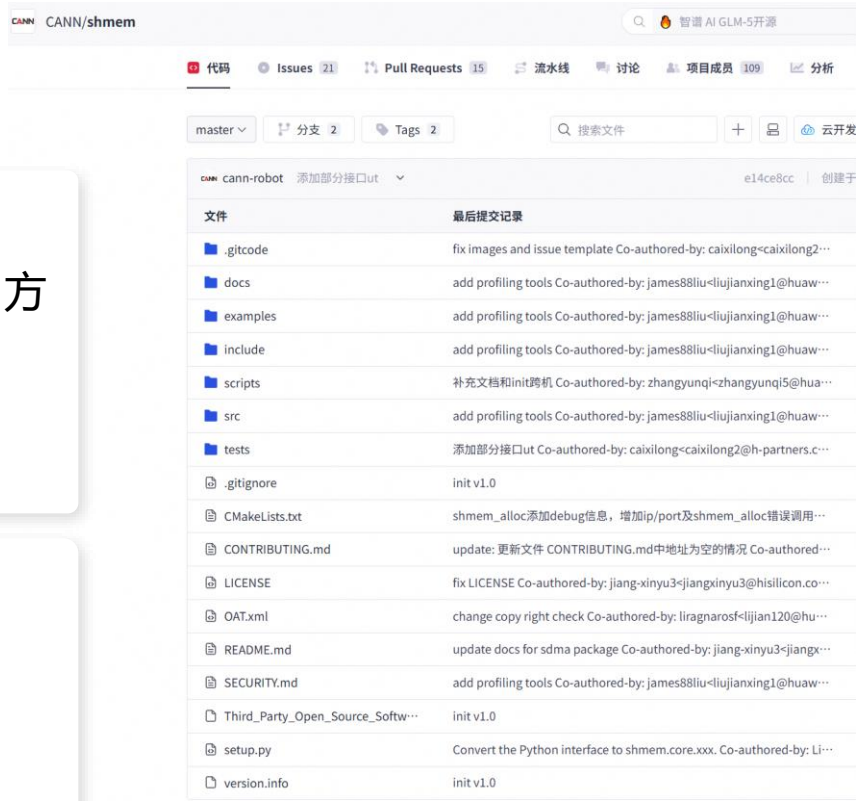
- PyPTO中封装了 OPENSHMEM 部分通信语义：

•shmem_put
•shmem_get
•shmem_signal
•shmem_wait
•shmem_barrier_all

- Triton distribute/torch/Hixl也已接入shmem能力

SHMEM社区介绍

https://gitcode.com/cann/shmem



SHMEM SIG

- 定位：Shmem SIG是Shmem的社区评议小组。Shmem相关需求、方案、投入、规划都在SIG组织讨论决策

运行机制

- 时间频率：每双周周三下午四点
- PR处理：常规特性PR每日例会评审，关键PR SIG例会评审
- issue处理：尽快提供有效答复，评估有价值后开发支持

CANN

https://gitcode.com/cann/catlass

代码仓地址： https://gitcode.com/cann/shmem

文档地址： https://shmem-doc.pages.dev

代码仓链接

SIG会议订阅链接

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann/shmem